Целочисленные типы данных: short, int и long

На этом уроке мы рассмотрим целочисленные типы данных в языке C++, их диапазоны значений, операцию деления, а также переполнение (что это такое и примеры).

Оглавление:

1. Целочисленные типы данных
2. Объявление целочисленных переменных
3. Диапазоны значений и знак целочисленных типов данных
4. Что используется по умолчанию: signed или unsigned?
5. Переполнение
6. Примеры переполнения
7. Деление целочисленных переменных

Целочисленные типы данных

Целочисленный тип данных — это тип, переменные которого могут содержать только целые числа (без дробной части, например: -2, -1, 0, 1, 2). В языке C++ есть 5 основных целочисленных типов, доступных для использования:

	Тип	Минимальный размер
Символьный тип данных	char	1 байт
Целочисленный тип данных	short	2 байта
	int	2 байта (но чаще всего 4 байта)
	long	4 байта
	long long	8 байт

> ***Примечание***: Тип char — это особый случай: он является как целочисленным, так и символьным типом данных. Об этом детально мы поговорим на одном из следующих уроков.

Основным различием между целочисленными типами, перечисленными выше, является их **размер**, чем он больше, тем больше значений сможет хранить переменная этого типа.

Объявление целочисленных переменных

Объявление происходит следующим образом:

```
char c;
short int s; // допустимо
int i; // предпочтительнее
int i;
long int li; // допустимо
long l; // предпочтительнее
long long int lli; // допустимо
long long lli; // предпочтительнее
```

В то время как полные названия `short int`, `long int` и `long long int` могут использоваться, их сокращенные версии (без `int`) более предпочтительны для использования. К тому же постоянное добавление `int` затрудняет чтение кода (легко перепутать с именем переменной).

Диапазоны значений и знак целочисленных типов данных

Как вы уже знаете из предыдущего урока, переменная с n-ным количеством бит может хранить 2^n возможных значений. Но что это за значения? Конкретные, которые находятся в диапазоне **Диапазон** — это значения от и до, которые может хранить определенный тип данных. Диапазон целочисленной переменной определяется двумя факторами: её размером (измеряется в битах) и её **знаком** (который может быть *signed* или *unsigned*).

Целочисленный тип signed (со знаком) означает, что переменная может содержать как положительные, так и отрицательные числа. Чтобы объявить переменную как signed, используйте ключевое слово `signed`:

```
signed char c;
signed short s;
signed int i;
signed long l;
signed long long lli;
```

По умолчанию, ключевое слово `signed` пишется перед типом данных.

1-байтовая целочисленная переменная со знаком (signed) имеет диапазон значений от -128 до 127, т.е. любое значение от -128 до 127 (включительно) может храниться в ней безопасно.

В некоторых случаях мы можем заранее знать, что отрицательные числа в программе использоваться не будут. Это очень часто встречается при использовании переменных для хранения количества или размера чего-либо (например, ваш рост или вес не может быть отрицательным).

Целочисленный тип unsigned (без знака) может содержать только положительные числа. Чтобы объявить переменную как **unsigned**, используйте ключевое слово `unsigned`:

```
unsigned char c;
unsigned short s;
unsigned int i;
unsigned long l;
unsigned long long lli;
```

1-байтовая целочисленная переменная без знака (unsigned) имеет диапазон значений от 0 до 255.

Обратите внимание, объявление переменной как unsigned означает, что она не сможет содержать отрицательные числа (только положительные).

Теперь, когда вы поняли разницу между signed и unsigned, давайте рассмотрим диапазоны значений разных типов данных:

Размер/Тип	Диапазон значений
1 байт signed	от -128 до 127
1 байт unsigned	от 0 до 255
2 байта signed	от -32 768 до 32 767
2 байта unsigned	от 0 до 65 535
4 байта signed	от -2 147 483 648 до 2 147 483 647
4 байта unsigned	от 0 до 4 294 967 295
8 байтов signed	от -9 223 372 036 854 775 808 до 9 223 372 036 854 775 807
8 байтов unsigned	от 0 до 18 446 744 073 709 551 615

Для математиков: Переменная signed с n-ным количеством бит имеет диапазон от $-(2^{n-1})$ до $2^{n-1}-1$. Переменная unsigned с n-ным количеством бит имеет диапазон от 0 до $(2^n)-1$.

Для гематематиков: Используем таблицу 😉

Начинающие программисты иногда путаются между signed и unsigned переменными. Но есть простой способ запомнить их различия. Чем отличается отрицательное число от положительного? Правильно! Минуса следует. Если минуса нет, значит число — положительное. Следовательно, целочисленный тип со знаком (signed) означает, что минус может присутствовать, т.е. числа могут быть как положительными, так и отрицательными. Целочисленный тип без знака (unsigned) означает, что минус спереди отсутствует, т.е. числа могут быть только положительными.

Что используется по умолчанию: signed или unsigned?

Так что же произойдет, если мы объявим переменную без указания signed или unsigned?

	Тип	По умолчанию
Символьный тип данных	char	signed или unsigned (в большинстве случаев signed)
Целочисленный тип данных	short	signed
	int	signed
	long	signed
	long long	signed

Все целочисленные типы данных, кроме char, являются signed по умолчанию. Тип char может быть как signed, так и unsigned (но, обычно, signed).

В большинстве случаев ключевое слово signed не пишется (оно и так используется по умолчанию).

Программисты, как правило, избегают использования целочисленных типов unsigned, если в этом нет особой надобности, так как с переменными unsigned ошибок, по статистике, возникает больше, нежели с переменными signed.

Правило: Используйте целочисленные типы signed, вместо unsigned.

Переполнение

Вопрос: «Что произойдет, если мы попытаемся использовать значение, которое находится вне диапазона значений определенного типа данных?». Ответ: «Переполнение».

Переполнение (англ. ***overflow***) случается при потере бит из-за того, что переменной не было выделено достаточно памяти для их хранения.

Данные хранятся в бинарном (двоичном) формате и каждый бит может иметь только 2 возможных значения (`0` или `1`). Вот как выглядит диапазон чисел от 0 до 15 в *десятичной* и *двоичной* системах:

Десятичная система	Двоичная система
0	0
1	1
2	10
3	11
4	100
5	101
6	110
7	111
8	1000
9	1001
10	1010
11	1011
12	1100
13	1101
14	1110
15	1111

Как вы можете видеть, чем больше число, тем больше ему требуется бит. Поскольку наши переменные имеют фиксированный размер, то на них накладываются ограничения на количество данных, которые они могут хранить.

Примеры переполнения

Рассмотрим переменную unsigned, которая состоит из 4 бит. Любое из двоичных чисел, перечисленных в таблице выше, поместится внутри этой переменной.

«Но что произойдет, если мы попытаемся присвоить значение, которое занимает больше 4 бит?». Правильно! Переполнение. Наша переменная будет хранить только 4 наименее значимых (те, что справа) бита, все остальные — потеряются.

Например, если мы попытаемся поместить число 21 в нашу 4-битную переменную:

Десятичная система	Двоичная система
21	10101

Число 21 занимает 5 бит (10101). 4 бита справа (0101) поместятся в переменную, а крайний левый бит (1) просто потеряется. Т.е. наша переменная будет содержать 0101, что равно 101 (нуль спереди не считается), а это уже число 5, а не 21.

Теперь рассмотрим пример в коде (тип short занимает 16 бит):

```
#include <iostream>
int main()
{
    unsigned short x = 65535; // допустимо: 65535 - максимальное значение, которое может хранить 16-битная unsigned переменная
    std::cout << "x was: " << x << std::endl;
    x = x + 1; // 65536 - это число больше максимально допустимого числа из диапазона допустимых значений. Следовательно, произ
    std::cout << "x is now: " << x << std::endl;
    return 0;
}
```

Результат выполнения программы:

```
x was: 65535
x is now: 0
```

Что случилось? Произошло переполнение, так как мы попытались присвоить переменной `x` значение больше, чем она способна в себе хранить.

> ***Для тех, кто хочет знать больше:*** Число 65 535 в двоичной системе счисления представлено как 1111 1111 1111 1111. 65 535 — это наибольшее число, которое может хранить 2-байтовая (16 бит) целочисленная переменная без знака, так как это число использует все 16 бит. Когда мы добавляем 1, то получаем число 65 536. Число 65 536 представлено в двоичной системе как 1 0000 0000 0000 0000, и занимает 17 бит! Следовательно, самый главный бит (которым является 1) теряется, а все 16 бит справа — остаются. Комбинация 0000 0000 0000 0000 соответствует десятичному 0, что и является нашим результатом.

Аналогичным образом, мы получим переполнение, использовав число меньше минимального из диапазона допустимых значений:

```
#include <iostream>
int main()
{
    unsigned short x = 0; // допустимо: 0 - минимальное значение, которое 2-байтовая unsigned переменная может хранить
    std::cout << "x was: " << x << std::endl;
    x = x - 1; // -1 - это число меньше минимально допустимого числа из диапазона допустимых значений. Следовательно, произ
    std::cout << "x is now: " << x << std::endl;
    return 0;
}
```

Результат выполнения программы:

```
x was: 0
x is now: 65535
```

Переполнение приводит к потере информации, а это никогда не приветствуется. Если есть хоть малейшее подозрение или предположение, что значением переменной может быть число, которое находится вне диапазона допустимых значений используемого типа данных — используйте тип данных побольше!

Правило: Никогда не допускайте возникновения переполнения в ваших программах!

Деление целочисленных переменных

В языке C++ при делении двух целых чисел, где результатом является другое целое число, всё довольно предсказуемо:

```
#include <iostream>
int main()
{
    std::cout << 20 / 4 << std::endl;
    return 0;
}
```

Результат:

```
5
```

Но что произойдет, если в результате деления двух целых чисел мы получим дробное число? Например:

```
#include <iostream>
int main()
{
    std::cout << 8 / 5 << std::endl;
    return 0;
}
```

Результат:

```
1
```

В языке C++ при делении целых чисел результатом всегда будет другое целое число. А такие числа не могут иметь дробь (она просто отбрасывается, **не округляется!**).

Рассмотрим детально вышеприведенный пример: `8 / 5 = 1.6`. Но как мы уже знаем, при делении целых чисел результатом является другое целое число. Таким образом, дробная часть (`0.6`) значения отбрасывается и остается `1`.

Правило: Будьте осторожны при делении целых чисел, так как любая дробная часть всегда отбрасывается.